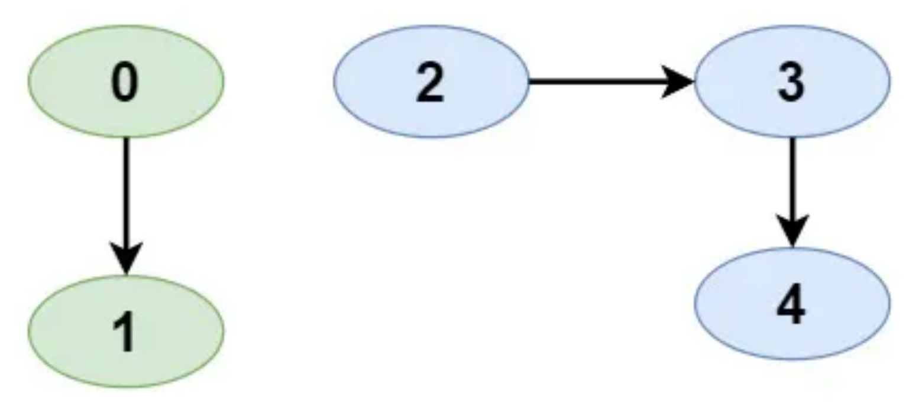
Solution: Number of Connected Components in an Undirected Graph

Problem Statement

Given an undirected graph represented by 'n' nodes labeled from `0` to `n-1` and a list of undirected edges (each edge is a pair of nodes), determine the number of connected components in the graph. A connected component is a group of nodes that are directly or indirectly linked to each other through the edges.

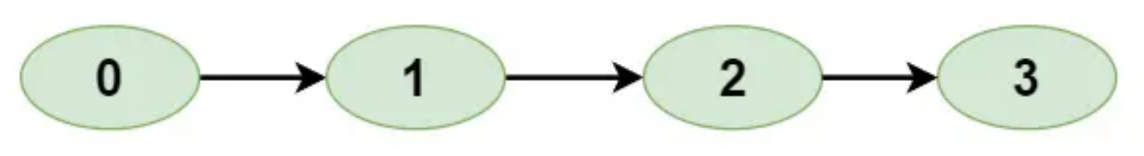
Examples

1. **Example 1**
   - **Input**: `n = 5`, `edges = [[0,1], [2,3], [3,4]]`



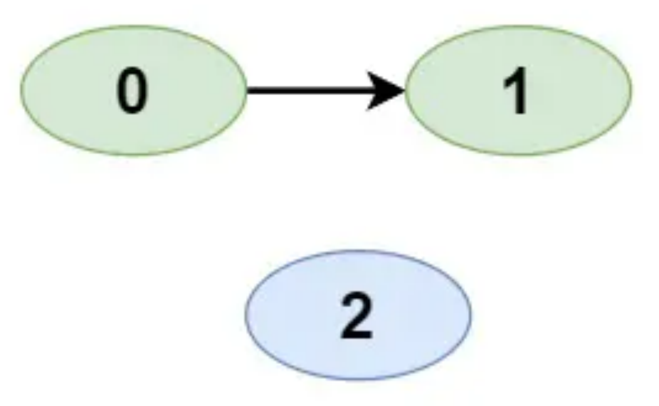
- **Expected Output**: `2`
- **Justification**: Two components are: `0-1`, and `2-3-4`.

2. **Example 2**
   - **Input**: `n = 4`, `edges = [[0,1], [1,2], [2,3]]`



- **Expected Output**: `1`
- **Justification**: All the nodes are connected in a single chain: `0-1-2-3`.

3. **Example 3**
   - **Input**: `n = 3`, `edges = [[0,1]]`



- **Expected Output**: `2`
- **Justification**: Two connected components exist: `0-1` and an isolated node `2`.

Solution

We will use the Union-Find (also known as Disjoint Set Union, DSU) data structure to solve this problem. This approach is efficient in identifying and merging different sets:

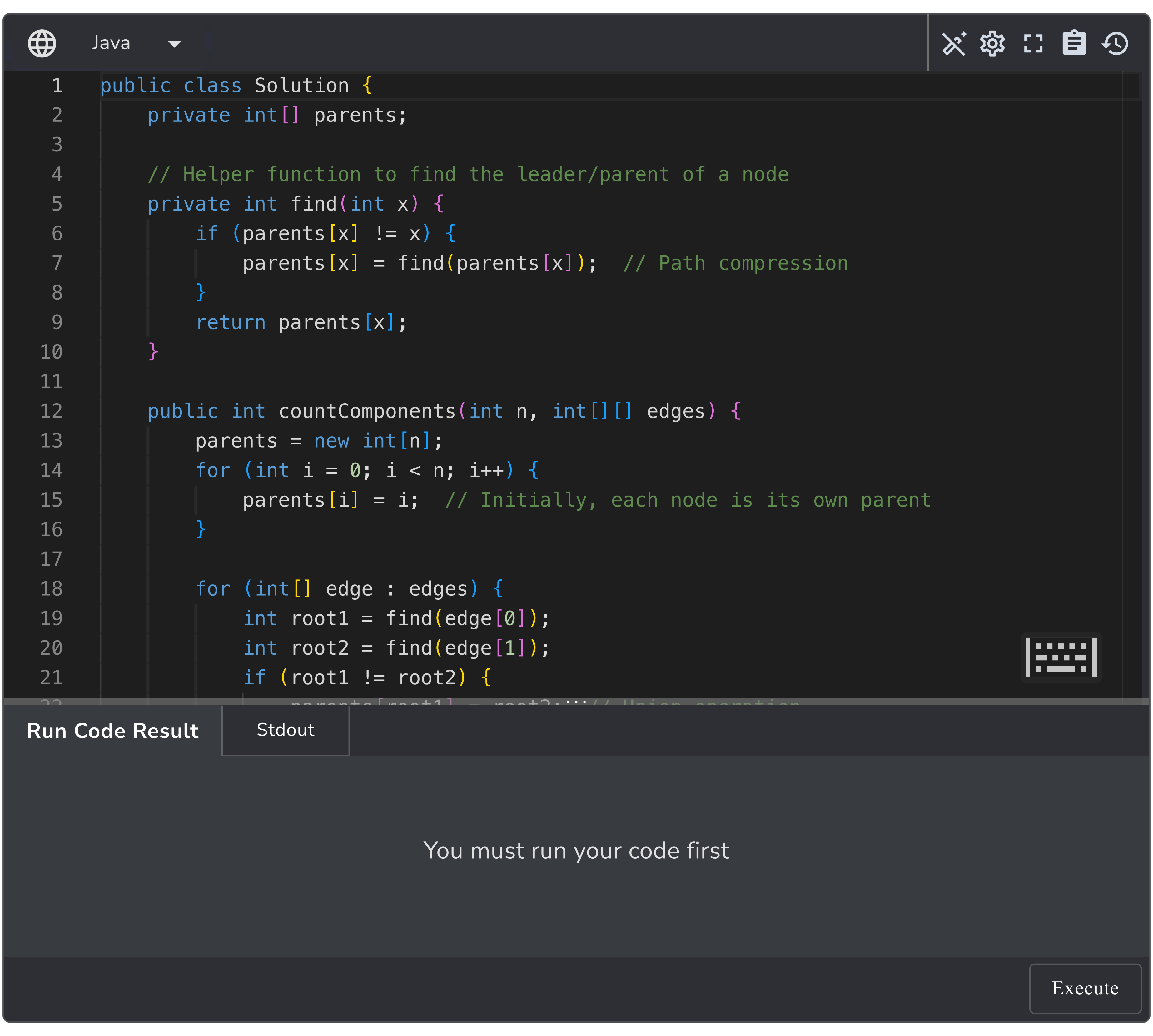
1. **Initialization**:
   - Create an array `parent` of size `n` where `parent[i] = i`. This array will keep track of the parent of each node. Initially, each node is its own parent.
   - Count the number of separate components. Initially, the count is `n` since every node is a separate component.
2. **Union Operation**:
   - For each edge `(u, v)`, find the parent of `u` and the parent of `v`.
   - If the parents are different, then `u` and `v` belong to different components. So, we merge them by assigning one's parent to the other and reduce the component count by 1.
3. **Find Operation**:
   - This operation determines the parent of a node. If a node's parent is itself, return the node. Otherwise, recursively find the parent of its parent. To optimize the search in future look-ups, we can apply path compression by updating the parent of the current node to its root during the recursion.
4. **Result**:
   - After processing all edges, the component count will represent the number of connected components.

Algorithm Walkthrough

Using the input `n = 5` and `edges = [[0,1], [2,3], [3,4]]`:

1. Initialize `parent = [0, 1, 2, 3, 4]` and `count = 5`.
2. Process edge `[0,1]`:
   - Parent of 0 is 0.
   - Parent of 1 is 1.
   - They have different parents. Merge them by setting parent of 1 to 0 and reduce count to 4.
3. Process edge `[2,3]`:
   - Parent of 2 is 2.
   - Parent of 3 is 3.
   - Merge them by setting parent of 3 to 2. Reduce count to 3.
4. Process edge `[3,4]`:
   - Parent of 3 is 2 (from previous merge).
   - Parent of 4 is 4.
   - Merge them by setting parent of 4 to 2. Reduce count to 2.
5. Final count is 2.

Code



```
public class Solution {
    private int[] parents;

    // Helper function to find the leader/parent of a node
    private int find(int x) {
        if (parents[x] != x) {
            parents[x] = find(parents[x]);  // Path compression
        }
        return parents[x];
    }

    public int countComponents(int n, int[][] edges) {
        parents = new int[n];
        for (int i = 0; i < n; i++) {
            parents[i] = i;  // Initially, each node is its own parent
        }

        for (int[] edge : edges) {
            int root1 = find(edge[0]);
            int root2 = find(edge[1]);
            if (root1 != root2) {
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The union operation is almost O(1) with path compression. So, for `E` edges, the time complexity is approximately O(E).
- **Space Complexity**: O(N) due to the `parent` array.